An FPGA-Based Secure and Privacy-Aware RISC-V SoC with a CNN Accelerator for Edge AI

Priyanshu Tyagi*
IIT Roorkee
priyanshu_t@ece.iitr.ac.in

Rhythm Patel*
SVNIT Surat
rhythmpatel01017@gmail.com

Sparsh Mittal
IIT Roorkee
sparsh.mittal@ece.iitr.ac.in

Rekha Singhal
TCS Research
rekha.singhal@tcs.com

***Abstract*—We propose a privacy-aware RISC-V SoC for edge inference, integrating a pipelined RV32IMC CPU, a quantized FPGA-based CNN accelerator, and a lightweight hardware security module. The accelerator employs parallel 8-bit MAC and max-pooling units. The design also features inter-layer pipelining in addition to intra-layer parallelism. This allows partial feature maps to be sent to the next layers without waiting for full completion. This approach significantly cuts down on inference latency. Moreover, we enforce security through custom CNN instructions embedding challenge–response authentication, while a noise injection unit ensures output privacy. Implemented on an Xilinx Zynq-7000 FPGA, the accelerator achieves inference latencies of 0.25 ms (MNIST) and 15.93 ms (CIFAR-10), delivering more than 10× speedup over the sequential baselines. The system delivers peak throughput of 10.56 GOPS and 2.48 GOPS, with energy efficiency reaching 60.3 TOPS/W and 10.3 TOPS/W for MNIST and CIFAR-10, respectively. The results demonstrate the viability of our work for secure edge AI workloads.**

I. Introduction

The ever-increasing growth of artificial intelligence (AI) on edge devices—from wearable health monitors to vision-enabled IoT systems—has underscored the need to accelerate AI algorithms on edge, while striking a balance between performance, energy efficiency, and data privacy. The RISC-V instruction set architecture (ISA) has become an attractive option for edge AI systems. By defining custom opcodes for CNN operations, RISC-V processors can trigger acceleration directly from the CPU pipeline, dramatically reducing CPU-accelerator communication overhead [1], [2]. For instance, RISC-V vector extension for machine learning kernels [3] delivers significant speedups over scalar implementations. [4] demonstrates a RISC-V coprocessor extension that offloads convolutions, boosting performance on CIFAR-10 classification tasks. Such implementations enable hardware reuse while retaining flexibility for diverse CNN variants. FPGAs complement this flexibility by offering parallelism, rapid prototyping, and a reconfigurable substrate for co-optimizing RISC-V control pipelines with accelerator datapaths. Prior FPGA-based accelerators [2], [5] demonstrate high throughput and efficiency while adapting to various CNN workloads. However, these designs generally operate as standalone modules controlled by external processors. They lack tight integration with CPU pipelines, which limits seamless edge deployment.

*Equal contribution. This work is supported by the Science and Engineering Research Board (SERB) India, under the project CRG/2022/003821. Priyanshu's research is supported by the Qualcomm Innovation Fellowship 2025.

Though performance and modularity are critical, most accelerators lack integrated security for privacy-sensitive deployment. Privacy concerns arise particularly for applications such as biometric authentication and smart health monitoring, which handle sensitive data. Without safeguards, model outputs or activations can reveal private information. In addition, recent research shows that edge AI systems are more vulnerable to inference-level attacks [6]–[8]. This can happen through direct access or adversarial side-channel attacks. Software techniques like differential privacy and encryption are too costly and complex for edge workloads. Hardware-based security techniques are lightweight and hence promising. Signature verification, challenge-response schemes, and noise injection mechanisms have shown promise in protecting AI inference pipelines. For example, [6] integrates instruction boundary protection and memory encryption into a CNN accelerator pipeline. [7] explores hardware acceleration of homomorphic encryption on FPGAs. [8] applies selective memory encryption to protect CNN models but incurs a significant latency penalty.

Other lightweight privacy methods, such as PUF-based authentication or noise injection, incur lower overhead but are rarely adopted in CNN accelerators. When used, they either operate in isolation as standalone hardware blocks or software-layer add-ons rather than as part of an integrated design. Most FPGA-based CNN pipelines still treat privacy as an external pre- or post-processing step, leaving a clear gap in achieving end-to-end secure inference within a unified SoC framework.

Research Gap Analysis and Motivation: Evidently, there is a dearth of works that combine high-performance CNN accelerators with RISC-V–based SoCs to support secure, real-time inference. Specifically, we find that:

- FPGA-based CNN accelerators lack integrated privacy controls or instruction-driven authentication [2], [5].
- RISC-V SoCs offer custom instruction pipelines but typically operate on unprotected inference triggers [3], [4].
- Security-centric CNN architectures focus on memory integrity and isolation, but are not designed for SoC-level integration with flexible inference logic [7], [9].
- There is a need to combine hardware-efficient authentication with output masking strategies for privacy protection.

To address this, we propose an edge-deployable RISC-V SoC architecture featuring a custom pipelined CNN accelerator, protected by a signature-verification FSM and a noise-

injection unit. This setup offers secure access control and output privacy with minimal overhead. Unlike prior CNN accelerators, our approach achieves tight hardware co-design of performance and privacy. The architecture achieves 10.68× and 10.22× speedups over the sequential baseline for MNIST and CIFAR-10, respectively, while sustaining energy efficiencies of up to 60.3 TOPS/W and 120 TOPS/mm^2.

II. Proposed System Overview

We propose a secure and privacy-aware RISC-V-based SoC, as shown in Fig. 1. It includes a pipelined RV32IMC CPU, a dedicated CNN accelerator, on-chip memory buffers, and modules for security and privacy. The RISC-V CPU executes standard instructions alongside a custom CNN instruction, which triggers the accelerator through a secure interface. The instruction includes the image pointer and authentication data, ensuring that inference occurs only after successful verification. Our architecture is optimized for secure and efficient edge inference on MNIST and CIFAR-10 datasets.

The CNN accelerator operates on pre-quantized weights and biases stored in on-chip BRAM buffers. It supports convolution, normalization, activation, pooling, and fully connected layers, with computation handled by two specialized Processing Elements (PEs). Intermediate feature maps are stored in local memory to reduce latency and off-chip data transfers. The Security and Privacy Module performs challenge–response authentication using signature verification logic, followed by optional differential noise injection into the CNN predictions for privacy preservation. The accelerator, CPU, and memory communicate over a Wishbone–to–AXI-Lite interconnect, enabling modular and scalable integration of system components. Overall, the design achieves tight coupling between general-purpose processing, CNN acceleration, and hardware-level privacy, making it suitable for secure edge AI deployments.

III. Proposed CNN Accelerator Architecture

Our CNN accelerator is a custom, fixed-function, fully synthesizable engine designed to perform quantized inference over MNIST and CIFAR-10 datasets. It supports up to 8-bit precision, enabling high area efficiency and throughput.

A. Processing Elements

We design the datapath of the accelerator using two complementary PE types: MAC (multiply–accumulate) and MP (max-pooling) PEs. These PEs form the fundamental hardware blocks that execute the arithmetic and comparison primitives required by CNNs. MAC PEs perform most of the computations, including 2D convolution, batch normalization (scale and shift), and FC layer operations. Pooling layers are mapped to MP PEs. This separation of roles enables precise hardware optimization and efficient parallelism across the accelerator.

Figure 2(a) shows the **MAC PE** architecture that carries out signed 8-bit MAC operations. The multiplication and accumulation stages are pipelined to deliver one result each clock cycle. Multiplications use DSP48E1 slices on the FPGA. Accumulation combines DSPs and registers to minimize latency

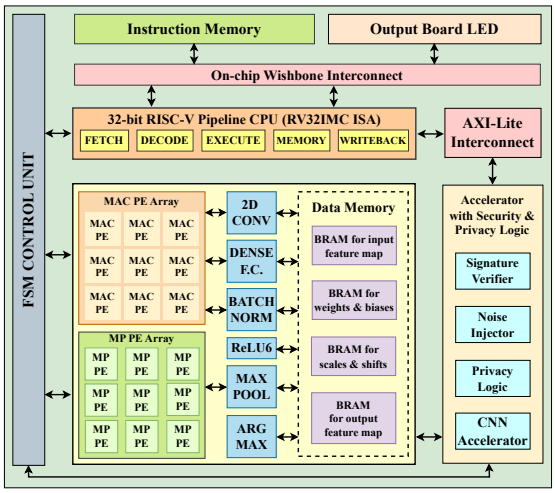


Fig. 1. Block diagram of proposed RISC-V SoC showing integrated CNN accelerator and security and privacy module

and area. Throughput improves when we balance the pipeline stages to avoid stalls. In our design, we implement three PE configurations: in the 1-PE setup, each MAC operation is processed sequentially; with 9 PEs, a 3×3 MAC block is computed in parallel; and with 128 PEs, the architecture is organized such that an entire output feature map channel is computed in parallel for all convolutional layers. This hierarchical scaling of parallelism improves throughput significantly. Since MAC operations dominate CNN workloads, optimizing MAC PE throughput and efficiency has a direct impact on inference latency and energy consumption.

The **MP PE** is a lightweight combinational unit for implementing 2×2 max-pooling operations, as shown in Figure 2(b). It compares four 4-bit inputs and outputs the maximum value. Multiple MP PEs operate in parallel, allowing simultaneous pooling across different feature map regions. The design is optimized for minimal logic depth to achieve very low latency.

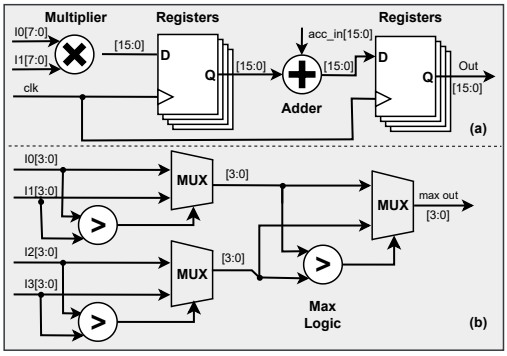


Fig. 2. Block diagrams of the proposed PEs: (a) MAC PE for 8-bit MAC operations, and (b) MP PE for 2×2 max pooling operations

B. Dataset-Specific CNN Architectures

For **MNIST**, the accelerator implements an 8-layer CNN for processing 28×28 grayscale images. The front-end consists of a 2D convolution producing a $26 \times 26 \times 32$ feature map, followed by ReLU and max pooling to reduce dimensions to $13 \times 13 \times 32$. A second convolution layer outputs $11 \times 11 \times 64$ features, again followed by ReLU activation and max pooling,

yielding a compact $5 \times 5 \times 64$ representation. This tensor is flattened into a 1×1600 vector, processed by two dense layers ($1600 \rightarrow 128$ and $128 \rightarrow 10$) and an argmax classifier.

For **CIFAR-10**, we use a deeper CNN for 32×32 RGB images. The architecture has several convolutional stages with same-padding to preserve consistent feature map alignment across layers, simplifying address generation for the accelerator. Each stage is followed by batch normalization and ReLU6 activation. We interleave max pooling layers to progressively reduce the size of the feature maps while increasing channel depth from 32 to 128. The final pooling layer outputs a $4 \times 4 \times 128$ feature map, which is flattened into a 1×2048 vector and passed through two dense layers ($2048 \rightarrow 128$ and $128 \rightarrow 10$) and an argmax classifier.

All convolution and dense layers use quantized 8-bit weights, biases, and batch-normalization scale and shift values obtained via Quantization-Aware Training to minimize accuracy loss. A centralized FSM sequences the computation linearly from input loading to final prediction. The network is fixed in hardware, with weights, biases, and batch-normalization parameters preloaded into dedicated on-chip memory. Weight buffers store filter coefficients, bias buffers hold per-channel bias values added to accumulations, and separate buffers hold batch-normalization scale and shift parameters applied before activation. Input images reside in BRAM, with a pointer from custom CNN instruction selecting an image to classify. This static parameter mapping enables aggressive pipelining and timing optimization during synthesis.

C. Pipelined Execution

Beyond intra-layer parallelism, the architecture incorporates inter-layer pipelining to reduce inference latency. Instead of waiting for a layer to complete before starting the next, partial outputs are forwarded as soon as they are ready. Normally, a $16 \times 16 \times 64$ feature map would be fully computed before proceeding. In the proposed design, once the subset of outputs needed to produce a feature for a channel is available, it is immediately stored in BRAM, and a control signal triggers computation in the subsequent layer. For example, as shown in Fig. 3, in a $16 \times 16 \times 64$ layer, once a 3×3 output region for all 128 channels is computed, it is stored in BRAM for the next layer to access without waiting for the remainder of the current layer, ensuring execution overlap without violating data dependencies [10].

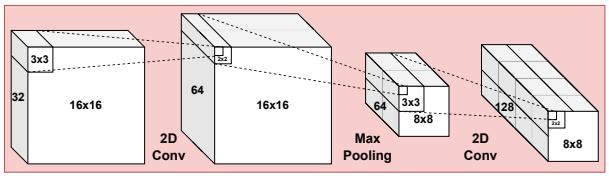


Fig. 3. Interlayer pipelining: early (partial) output sent to next-layer

For 2×2 max pooling, once all four contributing values for an output location are available, pooling is executed immediately and in parallel across multiple regions. This enables pooling layers to benefit from parallel execution once inputs are ready. The pipelined strategy is applied selectively across all layers, maximizing benefits between convolutional stages while pooling layers gain parallelism only when full input sets are available. Combined with scalable PE parallelism and early max pooling execution, this approach achieves high throughput with efficient resource utilization.

D. FSM-Controlled Execution Flow

The entire CNN accelerator is governed by a centralized FSM that begins with the **Idle State**, where the accelerator awaits a *cnn_start_cmd* signal from the CPU. In the **Input Load** stage, image data is fetched and stored in local feature map buffers. The FSM then proceeds to **Layer Computation**, where each CNN layer is executed sequentially by enabling the corresponding control signals for convolution, normalization, activation, and pooling operations; once a layer completes, control transitions to the next. After the final convolutional layer, the **Flattening Layer** converts the 2D feature maps into a 1D vector and passes them to the fully connected layers. Next, in the **Output Registering** stage, the final class scores are reduced using an argmax comparator to determine the predicted label. Finally, once the output is written to the result register, a *cnn_done* flag is raised to notify the CPU.

E. Memory Hierarchy and Data Flow

Our CNN accelerator makes use of a three-tiered memory hierarchy. First, **Input Buffers** store image data received from the CPU or RAM. Second, **Intermediate Buffers** hold the feature maps after each convolutional or pooling layer. These are implemented using BRAMs to support dual-port access and allow for pipelined read/write. Third, **Weight and Bias Buffers** are preloaded with quantized model parameters during SoC initialization and remain fixed throughout operation. All inter-layer communication is internal to the CNN engine, avoiding repeated accesses to main memory and reducing bandwidth pressure on the system interconnect. A global control unit arbitrates buffer access, manages BRAM addressing, and synchronizes read/write operations based on FSM state.

IV. RISC-V Core & Proposed Custom Instructions

The RISC-V processor is a 5-stage pipeline with forwarding, hazard detection, and custom instruction decoding (Fig. 4). We leverage RISC-V's flexibility to introduce custom instructions to control CNN execution from the CPU pipeline while embedding authentication metadata and privacy configuration.

Instruction Encoding: The proposed custom CNN instruction follows a 32-bit RISC-V format based on a modified I-type encoding scheme. Specific bit fields are reinterpreted to carry functional operands, authentication data, and control flags relevant to secure execution. Instruction fields are as follows: (i) A **7-bit opcode** distinguishes the instruction as a CNN operation. (ii) A **5-bit rd** shows the destination register, which stores the CNN Accelerator result. (iii) A **5-bit rs1** shows the source register, which contains the memory address (or pointer) to the input image. (iv) A **3-bit funct3** (abc) encodes privacy mode flags (a) and CNN variant (bc). Two unique modes are reserved—one for normal inference and

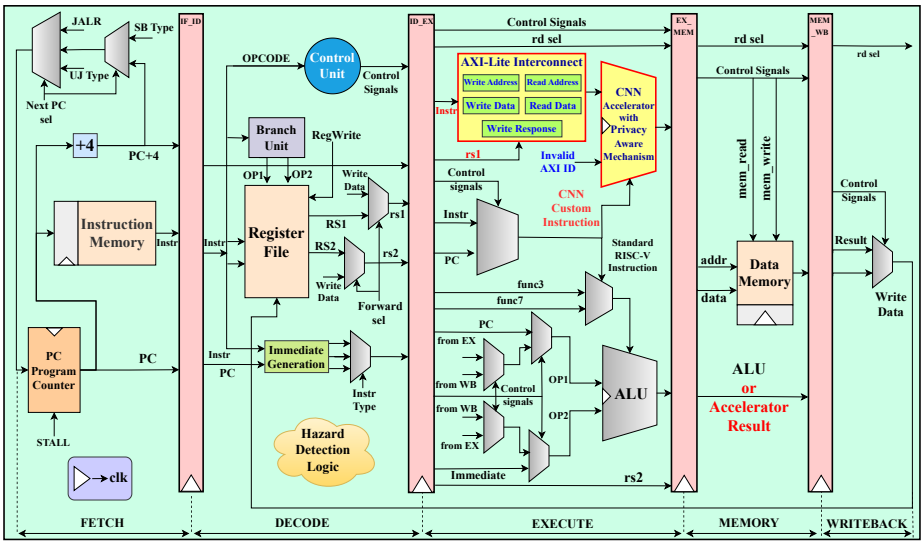


Fig. 4. Integration of proposed Privacy-Aware CNN Accelerator with RISC-V Core as an extension in the execute stage (shown in red)

another for inference with privacy mode enabled, as shown in Fig. 5. In privacy-enabled instructions, the MSB of the func3 is set to '1', which signals the output privacy module to inject noise into the Accelerator result. Otherwise, clean output is returned. (v) A **12-bit imm12** is reinterpreted to include a 4-bit secret key known to the CPU and the verification logic. It also contains a 4-bit random challenge issued by the CPU during runtime. Finally, there is a 4-bit signature, an XOR-based response computed offline and added to the instruction.

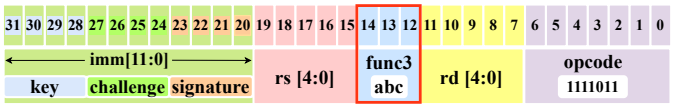


Fig. 5. Proposed custom instruction encoding with updated *func3*:*abc* = 000: MNIST, 100: Secure MNIST, 001: CIFAR-10, 101: Secure CIFAR-10; where 'a' = privacy mode, 'bc' = CNN variant

Instruction Decoding and Triggering: On encountering the opcode of the custom instruction, the instruction decoder reroutes the control flow to a dedicated CNN handler, which extracts the image pointer from rs1 and parses the 12-bit immediate field. It also generates control signals to initiate the CNN accelerator via an AXI-Lite command [11]. Finally, it sends authentication fields to the signature verification unit, monitors for completion signals (cnn_done), and handles the result. The instruction effectively performs a full inference cycle, from input specification to result retrieval. All this is done within a single programmatic line, while embedding security constraints at the hardware level [1].

V. Proposed Security and Privacy Features

Traditional CNN accelerators focus on performance and efficiency, but often leave vulnerabilities that can be exploited to leak sensitive data. The proposed SoC addresses this through two tightly hardware-enforced mechanisms: instruction-level authentication and privacy-based output obfuscation.

Threat Model and Assumptions: We consider a threat model where an adversary has software-level access to the system but cannot reprogram the FPGA fabric. This includes: (1) Malicious users attempting to invoke the CNN accelerator using unauthorized or modified instructions. (2) Adversaries trying to reverse-engineer or infer information from model outputs. (3) Side-channel attempts to exploit predictable outputs (e.g., class labels) for dataset reconstruction. We assume the model weights and architecture are fixed at synthesis and not accessible via software interfaces. Also, the adversary cannot physically tamper with BRAM or logic gates, but may attempt to manipulate registers or instruction memory.

A. Instruction-Level Authentication

We embed the first line of defense directly into the custom instruction itself. Every invocation of the CNN accelerator is coupled with a 12-bit authentication payload that includes a 4-bit secret key, a 4-bit runtime-generated challenge, and a 4-bit precomputed signature. The hardware verification unit re-derives this signature using simple XOR and bit-shift logic and compares it against the provided one. Only if this verification passes does the CNN accelerator operate. This ensures that: (1) Malicious or tampered instructions without a valid signature are denied execution. (2) CNN hardware remains idle unless explicitly authorized by the CPU. (3) Even if the custom instruction format is reverse-engineered, brute-forcing the correct combination of key, challenge, and signature requires 2^{12} attempts, which is infeasible under real-time constraints.

Figure 6 shows the proposed lightweight authentication logic (≈30 LUTs, 20 FFs) to stop unauthorized access to the CNN engine. It is governed by an FSM that interfaces directly with the CNN controller to either permit or block inference execution. It does so with three core states: (1) *IDLE*: Awaiting a new instruction. (2) *VERIFY*: Computing and comparing signature. (3) *UNLOCKED / ERROR*: Grant or deny access based on verification. This allows graceful degradation, such that if the authentication fails, the system does not crash, but only refuses to trigger inference.

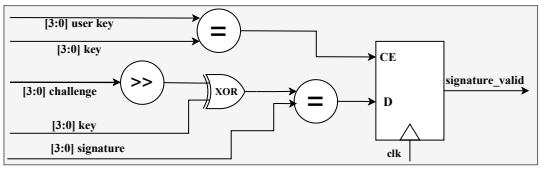


Fig. 6. Proposed lightweight authentication module illustrating signature verification for secure CNN inference

B. Secure Execution and Differential Output Privacy

Once authentication is successful and the FSM transitions to the *UNLOCKED* state, the CNN accelerator begins processing. Internally, it reads the image from memory, propagates it through the convolutional pipeline, and generates a final prediction. If privacy mode was enabled in the custom instruction (determined by the MSB of the func3), the predicted 4-bit class index is sent to the Differential Noise Injection module (as shown in Figure 7) before being returned to the CPU. This module uses an LFSR to generate a 4-bit pseudo-random mask, which is XORed with the prediction output. The privacy logic ensures that in the privacy mode: *final_output* = *predicted_class* $\oplus$ *lfsr_noise*. In the normal mode, *final_output* = *predicted_class*. This mechanism guarantees that even if an unauthorized instruction somehow bypassed the instruction memory, its output would be meaningless unless proper authentication and privacy configuration were met. Also, if privacy is not needed, accurate predictions are available.

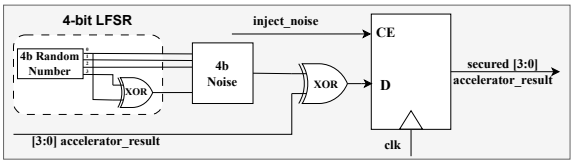


Fig. 7. Block diagram of the Differential Noise Injection module

Moreover, the proposed privacy method makes it difficult for adversaries to perform output-based attacks, such as (1) Membership inference, where adversaries try to infer if a sample was part of the training set. (2) Model inversion, which attempts to reconstruct input images from predicted class distributions, and (3) side-channel leakage, especially in settings where outputs are logged or observed in real-time. Our privacy approach introduces zero computational overhead on the CNN pipeline itself. It provides configurable privacy, which is controlled directly by the issuing instruction. Finally, it offers bit-level obfuscation, making reverse engineering harder without any performance penalty [12].

VI. Results & Discussion

We implement and deploy the complete SoC, including the RISC-V Processor, CNN Accelerator, Security and Privacy modules, on a ZedBoard Zynq-7000 Development Board. Owing to the purely digital implementation, our Hardware Accelerators achieve a similar accuracy as the software simulations, viz., 99.98% for MNIST and 99.18% for CIFAR-10.

A. Inference Latency & Speedup Analysis

The proposed accelerator achieves speedup through a combination of PE-level parallelism and selective pipelining. As discussed in Sections III-A and III-C, our design parallelism hierarchically: the 1-PE setup processes MAC operations sequentially; with 9 PEs, a 3×3 MAC block is computed in parallel; and with 128 PEs, an entire output feature map channel is computed concurrently. Combined with selective inter-layer pipelining and early execution of max pooling once its input set is ready, this parallelism yields substantial speedup gains. The system achieves speedups of 10.68× for MNIST and 10.22× for CIFAR-10 over a sequential FPGA baseline in which layers are executed one MAC at a time without pipelining or PE-level parallelism. Using this sequential baseline rather than a CPU implementation ensures that improvements stem solely from the proposed architectural optimizations.

TABLE I
Layer-wise Latency Analysis of MNIST Accelerator

Layer Types	Latency (ms) [9 PEs]		Latency (ms) [128 PEs]	
	Sequential	Pipelined	Sequential	Pipelined
Conv 3×3×32	1.45	0.32	0.12	0.02
MaxPool 2×2×32	0.41	0.09	0.56	0.06
Conv 3×3×64	15.72	3.45	0.81	0.08
MaxPool 2×2×64	0.11	0.03	0.08	0.01
Dense 1600→128	1.31	0.30	1.14	0.08
Entire Network	19.01	4.18	2.71	0.25

For the **MNIST dataset**, increasing parallelism to 9 PEs, pipelined execution already achieves an average speedup of 4.55×, as certain layers begin to hit memory bandwidth limits rather than compute bottlenecks and consistently outperforms sequential execution by reducing stalls and improving utilization. Table I reports latencies for the 9- and 128-PE configurations over five images. The 128-PE setup reduces the total latency from 2.71 ms to 0.25 ms, delivering a 10.68× speedup for the full network. This improvement stems from PE-level parallelism combined with both intra- and inter-layer pipelining, which together maximize resource utilization and minimize stalls across the MNIST inference flow.

TABLE II
Layer-wise Latency Analysis of CIFAR-10 Accelerator

Layer Types	Latency (μs) [9 PEs]		Latency (μs) [128 PEs]	
	Sequential	Pipelined	Sequential	Pipelined
3×3×3 Conv (32)	667.87	146.54	9.70	0.88
3×3×32 Conv (32)	7174.87	1599.67	102.12	9.37
MaxPool 32→16	443.95	99.83	6.45	0.59
3×3×32 Conv (64)	1446722	319386.53	17985.83	1873.97
3×3×64 Conv (64)	3358872.73	718413.25	40663.20	4215.23
MaxPool 16→8	220.77	53.83	3.08	0.26
3×3×64 Conv (128)	2364052	512120.46	30578.10	3004.79
3×3×128 Conv (128)	4727136.76	1072075.41	68423.23	6290.09
MaxPool 8→4	105.71	23	1.55	0.08
Dense 2048→128	380487.36	90620.64	5000	531.77
Entire Network	12365470.41	2714562.31	162774.46	15927.40

For the **CIFAR-10 dataset**, results in Table II demonstrate similar trends but with more pronounced gains for convolution-heavy layers. The latency reduces to 15.93 ms for the 128-PE configuration, achieving a 10.22× speedup over the sequential execution. Similar to the MNIST case, these improvements stem from PE-level parallelism combined with intra- and inter-layer pipelining. The deeper network structure of CIFAR-10 benefits more from pipelining, enabling higher sustained

throughput in large-PE configurations. Overall, the results confirm that the proposed accelerator scales effectively with workload complexity and parallel resources.

B. Resource Utilization Analysis:

We synthesize the proposed accelerator for three parallel configurations—1 PE, 9 PEs, and 128 PEs. Table III presents the Post-Implementation resource utilization of the proposed CNN accelerator across three PE configurations. The results confirm that the accelerator scales well with the number of PEs. Most of the extra resource usage comes from the replication of compute logic, specifically LUTs, FFs and PEs that use DSP slices. At the same time, BRAM usage is fixed. This indicates that compute parallelism, rather than memory, is the primary scalability driver in our design [1].

MNIST Accelerator: The LUT and the flip-flop (FF) count increase while going from the single-PE design to the 128-PE configuration, reflecting the replication of compute units and associated control logic. BRAM usage is fixed at 80 BRAM36 blocks for all configurations, owing to the pre-allocated and reusable memory mapping for weights, biases, and activations. DSP usage grows from 5 in the single-PE version to 29 in the largest configuration, proportional to the number of parallel MAC units. The maximum reported area is 0.088 mm^2, which remains within a small fraction of the available FPGA fabric.

TABLE III
RESOURCE UTILIZATION OF THE PROPOSED RISC-V SOC

Resources	MNIST Accelerator			CIFAR-10 Accelerator		
	1 PE	9 PEs	128 PEs	1 PE	9 PEs	128 PEs
LUTs	19,366	23,166	37,786	28,417	33,817	49,081
FFs	4,283	5,489	8,297	6,183	7,047	10,279
BRAM36	80	80	80	113	113	113
DSP48E1	5	7	29	5	11	47
Area (mm^2)	0.053	0.059	0.088	0.076	0.089	0.119

CIFAR-10 Accelerator: The CIFAR-10 design exhibits steeper DSP scaling (5 to 47) compared to MNIST, a direct result of wider convolution layers that can sustain more simultaneous MAC operations. LUTs and FFs usage grows more substantially as deeper layers require additional control and data-path logic to manage the higher degree of parallelism. BRAM usage is fixed at 113 BRAM36 blocks, driven by the larger weight and activation buffers required for the deeper network. The area rises to a maximum of 0.119 mm^2, still representing a modest portion of the available resources.

C. Security and Privacy Modules:

While offering two levels of protection, the proposed design introduces minimal resource penalty. Post-synthesis reports show that the signature verification logic uses $\sim$30 LUTs and 20 flip-flops. The LFSR-based noise injector uses $\sim$40 LUTs with a 4-stage feedback polynomial. Combined, these modules contribute less than 2% area overhead on Xilinx Zynq-7000 and introduce less than 1 cycle latency overhead in the prediction path. This overhead is acceptable given the security benefits, especially since both modules operate in parallel with the core CNN accelerator.

D. Power & Throughput Analysis

Table IV highlights that BRAMs and DSPs dominate the power consumption. For MNIST, BRAMs make up the largest share (40.3%) due to frequent access to weights and activations, and low compute demand. In the CIFAR-10 accelerator, DSPs consume high power (41.5%) due to deeper convolution layers that support higher parallel MACs. Contributions from LUT and FF remain low in both cases, under 10%, thus control logic and non-arithmetic datapaths have a minor role.

TABLE IV
COMPONENT-WISE ON-CHIP POWER CONSUMPTION

On-chip components	DSPs	BRAMs	LUTs	FFs	Others
MNIST Accelerator	27%	40.3%	5.5%	4.7%	22.5%
CIFAR-10 Accelerator	41.5%	31.5%	10%	4%	13%

Table V shows the hardware metrics of the proposed accelerators for the 128-PE configuration. Clearly, the accelerator consumes low energy by virtue of storing the weight, bias, and activation on-chip, which minimizes external memory access. Also, it achieves a high value of TOPS/W and TOPS/mm^2.

TABLE V
HARDWARE METRICS SUMMARY

Metrics	MNIST	CIFAR-10	Metrics	MNIST	CIFAR-10
Ops/Inference	2.64M	39.5M	**Latency**	0.25ms	15.93ms
Throughput	10.56 GOPS	2.48 GOPS	**Energy**	0.0438 mJ	3.84 mJ
Power	175mW	241mW	**TOPS/W**	60.3	10.3
Area	0.088 mm^2	0.119mm^2	**TOPS/mm^2**	120	20.8

VII. CONCLUSION

This paper presents a tightly integrated RISC-V SoC with an FPGA-based CNN accelerator, delivering high-performance inference. The design reduces latency and improves efficiency by utilizing pipelining and scalable parallelism in the PE. Meanwhile, signature-based authentication and noise injection ensure secure and privacy-preserving operations. The results prove that the proposed architecture offers a practical and scalable solution for secure edge AI deployment.

REFERENCES

[1] M. Ali *et al.*, "RISC-V based MPSoC design exploration for FPGAs: Area, power and performance," in *ISARC*, 2020, pp. 193–207.

[2] X. Wei *et al.*, "Automated Systolic Array Architecture Synthesis for High Throughput CNN Inference on FPGAs," in *DAC*, 2017.

[3] I. A. Assir *et al.*, "Arrow: A risc-v vector accelerator for machine learning inference," *arXiv preprint arXiv:2107.07169*, 2021.

[4] S. Zhang *et al.*, "A RISC-v based coprocessor accelerator technology research for convolution neural networks," in *JoP*, 2020.

[5] S. Basalama *et al.*, "Flexcnn: An end-to-end framework for composing cnn accelerators on fpga," *ACM TRTS*, 2023.

[6] W. Hua, M. Umar, Z. Zhang, and G. E. Suh, "Guardnn: Secure accelerator architecture for privacy-preserving deep learning," 2022.

[7] Z. Yang *et al.*, "FPGA-based hardware accelerator of homomorphic encryption for efficient federated learning," *arXiv:2007.10560*, 2020.

[8] P. Zuo *et al.*, "SEALing Neural Network Models in Encrypted Deep Learning Accelerators," in *DAC*, 2021, pp. 1255–1260.

[9] W. Hua *et al.*, "GuardNN: secure accelerator architecture for privacy-preserving deep learning," in *DAC*, 2022, p. 349–354.

[10] A. Garofalo *et al.*, "PULP-NN: Accelerating quantized neural networks on parallel ultra-low-power RISC-V processors," *PTRSA*, 2020.

[11] Z. Jiang *et al.*, "Towards a Real-Time AXI-Interconnect for Highly Integrated SoCs," *IEEE Transactions on Computers*, 2022.

[12] D. Desfontaines and B. Pejó, "Sok: differential privacies," *arXiv preprint arXiv:1906.01337*, 2019.